Week 1 Day 3 A. Soniya

Theory

1) A reference type stores the memory address (reference) of the actual value, not the value itself.

Reference Types:

- Object
- Array
- Function
- Date

Primitive Types:

- Number
- String
- Boolean
- null
- undefined
- Symbol
- BigInt

Primitive values are copied by value. Reference types are copied by reference.

2) Array is a special kind of Object.

typeof [] "object"

3) typeof [] → "Object"

typeof function () {} → "function"

typeof {} → "Object"

4) Primitives compare their values. Objects compare their

references (memory address).

5) const d = new Date();

6) Math.floor() → Round down

Math.ceil() → Round up

Math.max() → Returns largest value

7) Functions are first-class because they can

- be stored in variables
- be passed as arguments
- be returned from functions
- be stored inside objects

8.) Yes. Because the function receives the object's reference so modifying its properties changes the original object.

9) Array.isArray(variable)

10) For reference types, there is effectively no difference Both return true only if both operands refer to the same object.

Output Prediction

11) 2

12) false

13) object

14) object

15) function

16) true

17) false

18) 2025

19) 7

20) 4 5

Error Finding

21) 100 → Reason: Object mutation affects the original object because the reference is shared.

22) [] → Reason: string length = 0 removes all elements.

23) TypeError → Reason: newProp is undefined, so deep cannot be assigned.

24) true → Reason: The array is converted to the string "1,2" before comparison.

25) Invalid Date

26) -Infinity → Reason: No arguments means there is no maximum value to compare, so Math.max() returns -Infinity.

27) "" → An empty string

28) "[object Object]"

29) 1 → Reason: Object.freeze() prevents modification of existing properties.

30) false → Reason: They are two different function objects

Coding:

31) const book = {
    title: "Javascript Basics",
    auther: "John",

```
    Year : 2025,
};
Console.log(book);
```

32)
```
const arr = [1,2,3,4,5];
console.log(arr === arr);
const copy = [...arr];
console.log(arr === copy);
```
o/p : true
false

33)
```
function shallowCopy(obj) {
    return {...obj};
}
```

34)
```
const birthday = new Date(2002, 5, 20);
console.log(birthday.toLocaleDateString());
console.log(birthday.getDay());
```

35)
```
let result = Math.floor(Math.random() * 100) + Math.max(5, 10);
console.log(result);
```

36)
```
function isPlainObject(value) {
    return typeof value === "object" &&
           value !== null &&
           !Array.isArray(value);
}
```

37)
```
function copyArray(arr){
    return [...arr];
}
```

38)
```
const greet = function(name){
    return "Hello" + name;
};
```

```
function execute (fn) {
    console.log (fn("John"));
}
execute (greet);
```

39)
```
const obj = {
    user : {
        name : "John"
    }};
const clone = {
    user : {
        ... obj.user
    }};
```

40)
```
const obj = Object.freeze ({
    name : "John"
});
obj.name = "David";
console.log (obj.name);   O/P: John
```

41) Because modifying the cart inside the function changes the original object, which can lead to unexpected side effects.

42) Because === compares object references, not property values. Two separate objects with identical contents still have different references.

43) use the Date object.

Eg) new Date()

44) Use

math.trunc()

45) Use

Array.isArray(products)

46) Create a copy before modifying.

Eg):

const preview [...cart];

47) Use

Math.floor(Math.random() * 9000) + 1000

48)

The input string is not a valid date format, causing the date constructor to produce an Invalid Date Object.

49) Object.freeze() is shallow, so nested objects can still be modified unless they are also frozen.

Eg):
```
const config = Object.freeze({
    settings : {
      theme : "dark"
    }
});
config.settings.theme = "light"; // This change succeeds.
```

50) Because === compares references, not contents. Two arrays with identical elements are different objects.

Eg): [1,2] === [1,2] O/P : false

To compare contents, compare their elements (or serialize simple arrays):

JSON.stringify(arr1) === JSON.stringify(arr2)